

Understanding the Game of Life

The Game of Life by John Conway is a model built on very simple rules, but may lead to unpredictable and pretty random and chaotic results over time...

The rules for the game are as follows:

Underpopulation: Any live cell with fewer than two live neighbors dies, as if by loneliness.

Survival: Any live cell with two or three live neighbors lives on to the next generation.

Overpopulation: Any live cell with more than three live neighbors dies, as if by overcrowding.

Reproduction: Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

A CLASSIFICATION SYSTEM FOR 'LIFE' OBJECTS						
Characteristics of Life History				Class	Age	Example
Stable	Inactive		Stationary	Class I Still Lifes	∞	block
				Class II Oscillators	∞	blinker
	Active	Moving	Constant Bits	Class III Spaceships	∞	glider
			Increasing Bits	Class IV Glider Guns	∞	Glider Gun
Unstable	Predictable		Class V All Objects Not In Above (Known)	varies	n-ominoes	
	Unpredictable		Class VI All Objects Not In Above	?	broths	

Still lifes	
Block	
Beehive	
Loaf	
Boat	
Tub	

Oscillators	
Blinker (period 2)	
Toad (period 2)	
Beacon (period 2)	
Pulsar (period 3)	
Pentadecathlon (period 15)	

Spaceships	
Glider	
Light-weight spaceship (LWSS)	
Middle-weight spaceship (MWSS)	
Heavy-weight spaceship (HWSS)	

These images give some sort of reference as to what all patterns can emerge from basic patterns, as well as the nature of continuity of the original patterns chosen

Turing Completeness

A system is said to be turing complete if at all, in simple words, a computer can be made in it... For example people have built working computers inside minecraft, using the blocks and tools the game provides...

For a system to be turing complete, the following conditions must be met:

State (Memory): The ability to store information (essentially, 1s and 0s).

Transmission (Wires): The ability to move information from one location to another.

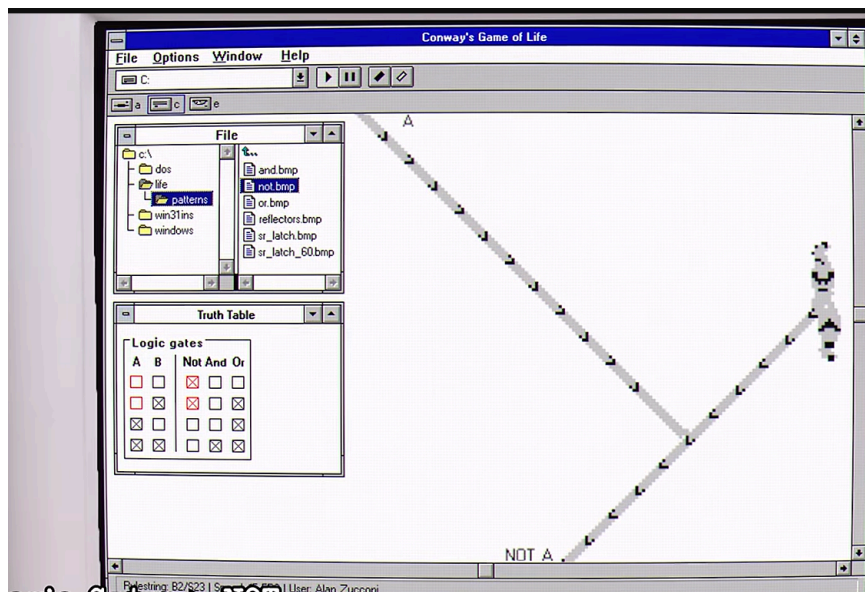
Universal Logic (Processing): The ability to perform basic logical operations.

Gliders are a very very essential pattern which enable the Game of Life to be turing complete...

A Glider Gospel gun can produce a glider, after every 30 or so states... and they can be used analogous to the binary bits that computers use

There are special patterns known as reflectors which can change the direction of the glider, and hence allow transmission... Input of one operation can be used as the output of the consequent or another independent operation

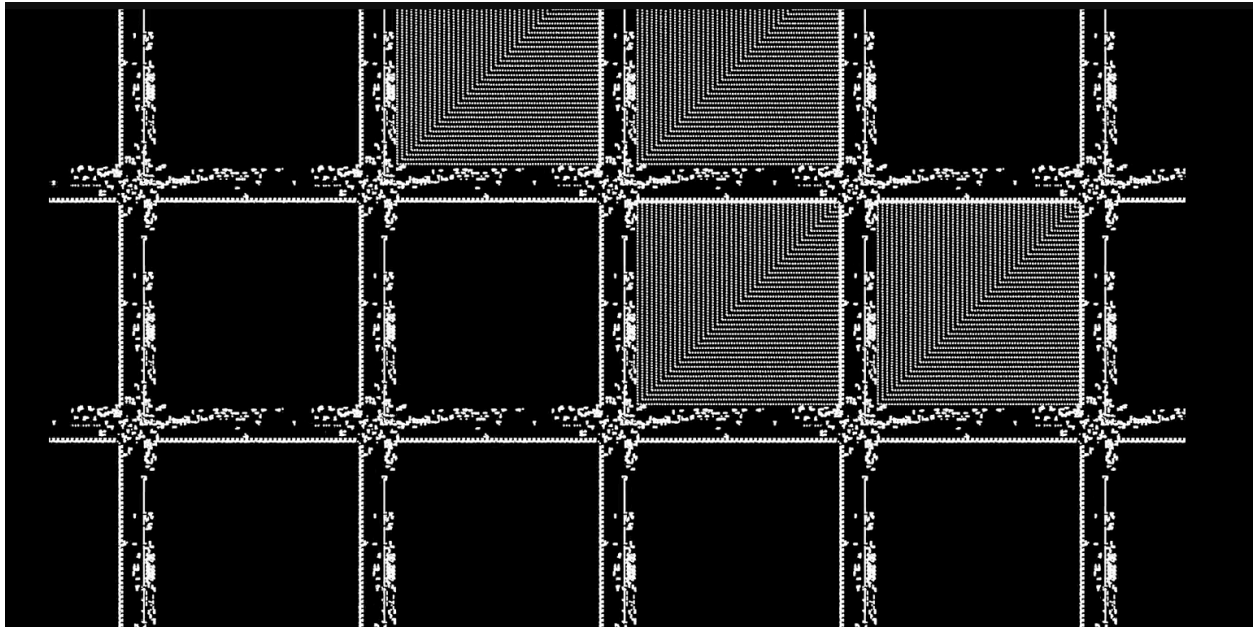
For logical operations, certain assemblies need to be created primarily with the help of gospel guns, gliders and glider eaters... and these assemblies enable the logical operands to exist, and operations to occur... Eg: AND, OR, NOR



Example of NOT A

Overall conclusion being... Game of Life is TURING COMPLETE

What this basically could mean is that, we can simulate the Game of Life, within Game of Life :))) which seems rather interesting to me personally



GAME OF LIFE BEING SIMULATED INSIDE GAME OF LIFE

Coming to coding implementation now

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as animation
4 N = 75
5 ON = 255
6 OFF = 0
7 grid = np.random.choice([ON, OFF], N*N, p=[0.08, 0.92]).reshape(N, N)
8 def update(frameNum, img, grid, N):
9     neighbors = (
10         np.roll(grid, 1, 0) + np.roll(grid, -1, 0) +
11         np.roll(grid, 1, 1) + np.roll(grid, -1, 1) +
12         np.roll(np.roll(grid, 1, 0), 1, 1) +
13         np.roll(np.roll(grid, -1, 0), -1, 1) +
14         np.roll(np.roll(grid, 1, 0), -1, 1) +
15         np.roll(np.roll(grid, -1, 0), 1, 1)
16     ) // 255
17
18     newGrid = np.copy(grid)
19
20     # Standard Conway Rules (B3/S23) - Much less chaotic than HighLife
21     newGrid[((grid == ON) & ((neighbors < 2) | (neighbors > 3)))] = OFF
22     newGrid[((grid == OFF) & (neighbors == 3))] = ON
23
24     img.set_data(newGrid)
25     grid[:] = newGrid[:]
26     return img,
27
28 fig, ax = plt.subplots()
29 ax.axis('off')
30
31 # 3. TWEAK: Stark black and white contrast instead of the 'magma' gradient
32 img = ax.imshow(grid, interpolation='nearest', cmap='binary')
33
34 # 4. TWEAK: Increased interval to 100ms to slow down the frame rate
35 ani = animation.FuncAnimation(fig, update, fargs=(img, grid, N),
36                               frames=300, interval=100, save_count=50)
37
38 plt.show()

```

OUTPUT:



:

T1 time

T2 time

